BUT SD

SAE Apprentissage pour l'IA

Cours n°2

Deep Learning avec Keras

Marc Métivier

IUT de Paris - Rives de Seine
Université Paris Cité

Deep Learning en Python

- La base : Python, Numpy, Jupyter
- Matériel : des GPU, des GPU et des GPU...
- Bibliothèques : deux mondes

Keras et TensorFlow

- Keras
  - API de haut niveau
  - MIT Licence
  - Dans TF depuis 2017
- TensorFlow (TF)
  - Framework le plus utilisé
  - Soutenue par Google
  - API de bas niveau
  - Apache Licence

PyTorch avec Lightning

- PyTorch Lightning
  - Interface pour PyTorch
  - API de haut niveau
  - Stable et propre
- PyTorch
  - Issue de la librairie Torch
  - Soutenue par Facebook
  - BSD Licence

Environnement de développement

Problème : chaque package a ses contraintes de versions

Solution : utiliser un *environnement Python*

- On crée un environnement avec Python 3.12 et les packages nécessaires
- On active cet environnement avant d'ouvrir Jupyter

Création d'un environnement :

- Choisir un nom d'environnement (par ex. `env_app_ia`)
- Ouvrir la fenêtre Anaconda Prompt
- Faire les commandes suivantes :
  - Créer un environnement : `conda create -n env_app_ia python=3.12`
  - Activer l'environnement : `conda activate env_app_ia`
  - Installation des packages (à faire dans cet ordre) :
    - `pip install tensorflow`
    - `pip install pandas matplotlib jupyter scikit-learn seaborn`

Utilisation de l'environnement

1. Ouvrir Anaconda Prompt

2. Se placer dans le dossier de travail

- Changer de lecteur : `Z:`
- Changer de dossier dans le lecteur : `cd chemin_de_dossiers/`

3. Activer l'environnement

```
conda activate env_app_ia
```

4. Lancer jupyter

```
jupyter notebook
```

Autre instructions utiles :

- Sortir de l'environnement : `conda deactivate`
- Supprimer un environnement : `conda env remove -n nom_env`

L'API Keras

API = "Interface de Programmation d'Application"

- Un cadre pour faciliter le développement de logiciels spécialisés

Keras : une API pour le Deep Learning

- Un cadre général pour le Deep Learning en Python
- Programmation orientée Objets
- Construction facilité de réseaux de neurones
- Des archictures complexes et avancées
- Entrainement et évaluation
- Elle fait maintenant partie de TensorFlow (Google)

Documentation

- Keras : https://keras.io/
- Documentation : https://keras.io/api/

Préparation des données

Le sous-module `keras.utils`

- Il contient quelques fonctions utiles pour le traitement des données
  - Fonction `split_data()` pour séparer les données
  - Fonction `to_categorical()` pour encoder en "One Hot Encoding"

Le sous-module `keras.datasets`

- Il contient différents ensemble de données classiques (MNIST, ...)
  - Chaque ensemble de données possède son propre module
- Chaque module contient une fonction `load_data()`
  - Elle retourne des données déjà séparées
  - Typiquement : `(X_train, y_train), (X_test, y_test)`

Création du modèle

La classe `Model`

- Une super-classe pour tous les types de modèles
- Toutes les classes de Modèle étendent cette classe
- Le sous-module `keras.models` contient des modèles prédéfinis

Créer un modèle "séquentiel"

- La classe `Sequential` : un enchaînement de couches
  - Couches définissant des entrées
  - Couches de neurones
  - Couches de traitements spécifiques

```
model = keras.models.Sequential()
model.add(keras.layers.Input(...))
model.add(keras.layers.Dense(...))
...
```

Pour créer un modèle plus complexe : étendre la classe `Model`

Couches de traitements (layers)

Le sous-modules `keras.layers` propose des couches prédéfinies

- Il existe de nombreux types de couches : voir https://keras.io/api/layers/

Couches utiles pour un perceptron

- Entrées : `Input(input_shape)`
- Applatissement des données : `Flatten()`
- Réorganisation : `Reshape(new_shape)`
- Couche de neurones "dense" : `Dense(nb_neurones, activation)`
  - Activation par défaut : pas de modification des sorties des neurones
  - Autres activations : "relu", "tanh", "sigmoid", "softmax"...

Compilation du modèle

La compilation sert à préparer le modèle pour l'entrainer

La méthode `compile()`

```
model.compile(optimizer, loss, metrics)
```

- `optimizer` :
  - Avec des paramètres prédéfinis : "sgd", "rmsprop", "adam", ...
  - Avec des paramètres à fixer :
    - `optimizer = keras.optimizers.SGD(learning_rate=0.01)`
    - voir https://keras.io/api/optimizers/
- `loss` : la fonction de coût
  - Il existe de nombreuses fonctions : voir https://keras.io/api/losses/
  - Classification : "categorical_crossentropy", "sparse_categorical_crossentropy", ...
  - Régression : "mean_squared_error", "mean_absolute_error", ...
- `metrics` : une liste des mesures à stocker pendant l'entraînement
  - Ex : "loss", "accuracy", "mse" (mean squared error), "mae", ...

Entrainement du modèle

Méthode `fit()`

```
s = model.fit(x, y, batch_size, epochs, verbose, validation_data)
```

- `x` : l'ensemble des données d'entrées pour l'entraînement
- `y` : l'ensemble des données de sortie pour l'entraînement
- `batch_size` : la taille des "batchs"
- `epochs` : le nombre d'étapes d'apprentissage
- `verbose` : types d'affichage d'informations
  - 0: pas d'affichage
  - 1: barres de progression
  - 2: une ligne par étape
- `validation_data` : un tuple avec les deux ensembles de tests
- La variable en sortie (`s`) reçoit un objet `History` avec un attribut `history`
  - `s.history` contiendra un `dict` avec les listes de mesures (celles de metrics)

Evaluation et utilisation du modèle

Méthode `evaluate()`

```
score = model.evaluate(x, y, batch_size, verbose)
```

- Les paramètres ont le même sens que dans `fit()`
- Cette méthode fait un cycle similaire à `fit()` mais sans apprentissage
- La sortie est : soit le coût (loss), soit une liste de mesures (celles de `metrics`)

La méthode `predict()`

```
y_predictions = model.predict(x, batch_size, verbose)
```

- Les paramètres ont le même sens que dans `fit()`
- Cette méthode sert à utiliser le modèle pour prédire des sorties
  - Elle retourne les prédictions du modèle par rapport aux entrées
  - Il n'y a pas d'apprentissage

Quelques méthodes utiles

Observer la structure d'un modèle : la méthode `summary()`

```
model.summary()
```

- Des paramètres permettent de spécifier des contraintes d'affichage

Sauvergarder un modèle : la méthode `save()` ou la fonction `save_model()`

```
# Deux solutions équivalentes
model.save(fichier)
keras.saving.save_model(model, fichier)
```

- `fichier` : un nom de fichier avec son chemin qui doit se terminer par ".keras"

Charger un modèle : la fonction `load_model()` du sous-module `saving`

```
keras.saving.load_model(fichier)
```